

# Proyecto Final – Fase 2

## Universidad de Guadalajara – Centro Universitario de Ciencias Exactas e Ingenierías

### De Arquitectura De Computadoras

Oscar Emiliano Ramirez Perez

Alondra Estefania Hernández Vargas

## 1. Introducción

El presente reporte documenta el desarrollo y la implementación de la Fase 2 del Proyecto Final, cuyo objetivo es el diseño del "single datapath" de un procesador MIPS de 32 bits capaz de ejecutar instrucciones tipo I. En esta etapa del proyecto, la arquitectura transita de un modelo puramente monociclo hacia una estructura segmentada, lo cual hace obligatoria la implementación de los registros intermedios o *buffers* de pipeline. A lo largo del documento se describe el código desarrollado en Verilog, la interconexión estructural de los módulos y las justificaciones teóricas de los nuevos componentes integrados.

## 2. Implementación de Hardware para Instrucciones Tipo I

Para dotar al datapath de la capacidad de ejecutar instrucciones con operandos inmediatos (tales como lw, sw, beq, ori y addi), fue indispensable añadir lógica adicional en la fase de decodificación y ejecución:

- **Extensor de Signo (Sign Extend):** Las instrucciones tipo I contienen un valor inmediato de 16 bits. Dado que la ALU y los buses de datos operan con 32 bits, se implementó un módulo en Verilog que aplica la técnica de replicación y concatenación. Este módulo copia el bit más significativo (el bit de signo, posición 15) dieciséis veces hacia la izquierda, preservando la magnitud matemática en complemento a dos del número original al convertirlo de 16 a 32 bits.
- **Multiplexor ALUSrc:** Se instanció un multiplexor dedicado en la entrada B de la ALU. Este componente permite a la Unidad de Control enrutar dinámicamente los datos: selecciona el segundo registro leído (ReadData2) para operaciones tipo R, o el valor del *Sign Extend* para las operaciones aritméticas y de memoria tipo I.
- **Multiplexor RegDst:** Para manejar la discrepancia en la ubicación del registro destino entre las instrucciones, este multiplexor utiliza la señal RegDst para elegir entre el campo rd (bits 15-11, usado en tipo R) y el campo rt (bits 20-16, usado en lw e i-types).

### 3. Segmentación del Cauce (Pipeline) y Buffers

Siguiendo las metodologías descritas en arquitecturas MIPS modernas, el datapath fue dividido en cinco etapas síncronas. Para evitar condiciones de carrera y pérdida de datos, se programaron registros de aislamiento (buffers) que se actualizan mediante un reloj común (clk):

1. **Buffer IF/ID (Instruction Fetch / Decode):** Aísla la memoria de instrucciones del decodificador, capturando la instrucción actual y el cálculo del Program Counter ( $PC + 4$ ).
2. **Buffer ID/EX (Decode / Execute):** Es el registro más denso de la arquitectura. Propaga los valores leídos del banco de registros, el inmediato extendido y, fundamentalmente, todas las señales de control necesarias para las etapas subsecuentes.
3. **Buffer EX/MEM (Execute / Memory):** Guarda los resultados aritméticos calculados por la ALU, la bandera Zero y el dato de escritura en memoria, preparándolos para la fase de acceso a la Memoria de Datos.
4. **Buffer MEM/WB (Memory / Write-Back):** Sostiene tanto el dato recién recuperado de la RAM (vital para la instrucción lw) como el resultado de la ALU, permitiendo que el multiplexor final seleccione cuál se escribirá definitivamente en el Banco de Registros.

### 4. Desarrollo Estructural en Verilog (DPTR)

El módulo principal (Top-Level) se refactorizó completamente. Se adoptó una arquitectura estructural basada en la instanciación de submódulos. La complejidad del ruteo de cables se gestionó nombrando las señales con prefijos correspondientes a su etapa activa (id\_, ex\_, mem\_, wb\_). Esta técnica garantizó que señales críticas como RegWrite y WriteData viajen a través de todo el cauce y solo realicen modificaciones en el Banco de Registros durante la quinta etapa del ciclo (Write-Back), asegurando la coherencia de los datos en ejecución.

### 5. Conclusiones

Oscar Ramirez: La implementación de los buffers y instrucciones tipo I fueron sumamente desafiantes para implementarlo, debido a que lograr comprender la lógica detrás de esto suele complicar las cosas, ya que es necesario entender que estamos haciendo antes de intentar implementar ya en código.

Alondra Hernández: Esta fase me pareció muy complicada y a la vez interesante porque cada vez me acerco mas a lo que es un CPU y el hardware, me gusto hacer esta fase por eso, cada vez entiendo mejor las cosas.

### 6. Referencias Bibliográficas

1. Patterson, D. A., & Hennessy, J. L. (2013). *Computer Organization and Design: The*

- Hardware/Software Interface* (5th ed., MIPS Edition). Morgan Kaufmann.  
(Específicamente el Capítulo 4.6 referente a Datapath y Pipeline).
2. Harris, D., & Harris, S. (2012). *Digital Design and Computer Architecture* (2nd ed.). Morgan Kaufmann. (Referencia estándar para el diseño de procesadores MIPS a nivel de transferencia de registros en Verilog).
  3. IEEE Standard for Verilog Hardware Description Language. (2006). IEEE Std 1364-2005. (Estándar técnico oficial para las sintaxis de replicación de bits, concatenación y módulos síncronos).